

UKRAINIAN CATHOLIC UNIVERSITY

FACULTY OF APPLIED SCIENCES

BUSINESS ANALYTICS & COMPUTER SCIENCE PROGRAMMES

Compressing images using learnable wavelets

Linear Algebra final project report

Authors:

Andrii KRYVYI

Nikita LENYK

Ostap KOSTIUK

Luka KONOVALOV

28 April 2026



APPLIED
SCIENCES
FACULTY ●

Abstract

This project studies image compression using learnable orthogonal wavelet transforms. Classical wavelet-based compression methods rely on predefined filters and fixed decomposition schemes, while our approach learns both the wavelet filters and the decomposition structure from data. We parameterize orthogonal wavelet filters through a sequence of rotations and delays in the polyphase representation, which guarantees perfect reconstruction and makes the filters differentiable with respect to learnable angle parameters. For two-dimensional images, we extend the transform using a tree-structured wavelet decomposition, where different subbands may be further decomposed or discarded.

The optimization is split into two parts. For a fixed tree structure, we optimize the wavelet filter parameters using the Adam optimizer to minimize reconstruction error after coefficient projection. To search over possible decomposition trees and coefficient keep/drop decisions, we use simulated annealing, with the reconstruction quality measured by PSNR. We train and evaluate the method on face images from the CelebA dataset and compare the resulting rate-distortion behavior with JPEG2000. The results show that learnable wavelets can produce compact representations specialized to the training distribution, although standard codecs such as JPEG2000 remain stronger when full quantization and entropy coding pipelines are taken into account.

1 Introduction

The goal of this project is to develop a learnable wavelet transform that produces compact signal representations. Instead of using predefined wavelet filters, we learn the transform directly from data.

Wavelet transforms decompose a signal into approximation and detail components across multiple scales. If the representation is sparse, most coefficients are close to zero, which allows efficient compression by discarding insignificant values.

Given a set of training images and a target compression rate, our goal is to learn an orthogonal wavelet-based transform that produces sparse image representations. More precisely, we seek a wavelet decomposition tree, a set of orthogonal wavelet filters, and a coefficient projection strategy such that, after transforming an image, discarding part of its coefficients, and reconstructing it with the inverse transform, the reconstruction error is as small as possible.

Formally, for an image X , we apply a transform P defined by a tree structure and learned wavelet filters, then apply a projection M that removes selected coefficients, and finally reconstruct the image using P^{-1} . The objective is to minimize the reconstruction loss

$$\|P^{-1}MPX - X\|_F^2$$

under a fixed compression constraint. In this setting, better compression corresponds to keeping fewer coefficients while preserving high reconstruction quality, measured mainly by PSNR.

In this work, we parameterize the wavelet filters, wavelet decomposition tree, and optimize them from data using two algorithms: Adam for optimizing filter parameters and simulated annealing for optimizing tree structure.

2 Module over z-domain

Definition 2.1 (Z-domain). To work with wavelets it is useful to define z-domain which is a ring $\mathbb{R}[z, z^{-1}]$ of Laurent polynomials of the following form:

$$a(z) = \sum_{k=-\min}^{\max} a[k]z^{-k}$$

There is a one-to-one mapping between real finite discrete signals and such Laurent polynomials. In z-domain convolution between two signals becomes just polynomial multiplication.

Definition 2.2 (Adjoint polynomial). For each polynomial $a(z)$ define an adjoint polynomial as $\tilde{a}(z) = a(z^{-1}) = \sum_{k=-\max}^{\min} a[-k]z^{-k}$. Adjoint polynomial corresponds to the signal a reversed.

To take further we also define a module (which is a generalization of a vector space, but instead of fields, we work over rings) over z-domain. We work with 2-dimensional vectors of Laurent polynomials:

$$a(z) = \begin{bmatrix} a_1(z) \\ a_2(z) \end{bmatrix}$$

Definition 2.3 (Paraunitary transpose). A paraunitary transpose is $A^\sharp = \tilde{A}^T$, that is take adjoint polynomial to each element and take a transpose of that matrix/vector.

Definition 2.4 (Paraunitary matrix). A matrix A is called paraunitary iff $A^\sharp(z)A(z) = I$.

Vaidyanathan[5] has shown that any orthogonal pair of filters with perfect reconstruction can be represented using a 2x2 paraunitary matrix (called polyphase matrix):

$$P(z) = \begin{bmatrix} h_e(z) & g_e(z) \\ h_o(z) & g_o(z) \end{bmatrix}$$

With $h_e(z)$ and $h_o(z)$ being Laurent polynomials such that

$$h(z) = h_e(z^2) + zh_o(z^2) \tag{1}$$

And same for g_e, g_o . Note that basically h_e is a polynomial related to the filter when we consider only even-positioned elements and h_o for odd-positioned elements.

3 Discrete wavelet transform

From now on we define the discrete wavelet transform (DWT) via a filter bank scheme. It is defined by two filters: a low-pass filter $h \in \mathbb{R}^m$ and a high-pass filter $g \in \mathbb{R}^m$ that are constrained to form an orthonormal basis by shifting them. The inverse filter-bank is

defined by two filters $\tilde{h}$ and $\tilde{g}$, which are reversed versions of h and g respectively. Note that their laurent polynomials are adjoint to the laurent polynomials of h and g (refer to section 2).

1D case. Let $x \in \mathbb{R}^N$ be a discrete signal. Denote by $*$ the discrete convolution operator; by D the downsampling operator by a factor of 2; U the upsampling operator and R_i the circular shift operator by i positions:

$$(D(x))_k = x_{2k} \quad U(x)_k = \begin{cases} x_{k/2}, & \text{if } k \text{ is even} \\ 0, & \text{if } k \text{ is odd} \end{cases} \quad (R_i x)_k = x_{(k-i) \bmod N}$$

One level wavelet transform works by first applying convolution between the signal and the low-pass and high-pass filters, followed by downsampling:

$$a = D(h * x), \quad d = D(g * x),$$

where a are approximation coefficients and d are detail coefficients.

The inverse is defined by upsampling the approximation and detail coefficients and convolving with the corresponding filters:

$$x = U(a) * \tilde{h} + U(d) * \tilde{g}$$

The filters h and g are chosen such that the following set forms an orthonormal basis of $\mathbb{R}^N$:

$$\mathcal{B} = \{R_{2i}h\}_{i=0}^{N/2-1} \cup \{R_{2i}g\}_{i=0}^{N/2-1}$$

2D case. Let $X \in \mathbb{R}^{N \times N}$ be a discrete image. Denote by $*$ the 2D convolution operator; by D the downsampling operator by a factor of 2 and $R_{i,j}$ the circular shift operator by i positions:

$$(D(X))_{k,m} = X_{2k,2m} \quad U(X)_{k,m} = \begin{cases} X_{k/2,m/2}, & \text{if } k \text{ and } m \text{ are even} \\ 0, & \text{otherwise} \end{cases}$$

$$(R_{i,j}X)_{k,m} = X_{(k-i) \bmod N, (m-j) \bmod N}$$

$$\begin{aligned} \text{LL} &= D(hh^T * X), & \text{LH} &= D(hg^T * X) \\ \text{HL} &= D(gh^T * X), & \text{HH} &= D(gg^T * X) \end{aligned}$$

This gives four smaller matrices, each having side twice smaller than before.

The inverse is defined by upsampling the subbands and convolving with the corresponding filters:

$$X = U(\text{LL}) * \tilde{h}\tilde{h}^T + U(\text{LH}) * \tilde{h}\tilde{g}^T + U(\text{HL}) * \tilde{g}\tilde{h}^T + U(\text{HH}) * \tilde{g}\tilde{g}^T$$

The corresponding orthonormal basis for $\mathbb{R}^{N \times N}$ is as follows:

$$\begin{aligned} \mathcal{B} &= \{R_{2i,2j}(hh^T)\}_{i,j=0}^{N/2-1} \cup \{R_{2i,2j}(hg^T)\}_{i,j=0}^{N/2-1} \\ &\cup \{R_{2i,2j}(gh^T)\}_{i,j=0}^{N/2-1} \cup \{R_{2i,2j}(gg^T)\}_{i,j=0}^{N/2-1} \end{aligned}$$

4 Parametrization of orthogonal wavelets

Vetterli and Le Gall[6] has shown that polyphase matrices for the pair of filters of length $2K$ can be decomposed into a sequence of delays and elementary matrices:

$$P(z) = \begin{bmatrix} 1 & \alpha_0 \\ -\alpha_0 & 1 \end{bmatrix} \prod_{k=1}^{K-1} \begin{bmatrix} 1 & 0 \\ 0 & z^{-1} \end{bmatrix} \begin{bmatrix} 1 & \alpha_k \\ -\alpha_k & 1 \end{bmatrix}$$

We also assume wavelets that preserve energy:

$$\det P(z) = z^{-k} \quad (2)$$

Lemma 4.1. *For any 2×2 paraunitary matrix $P(z)$ with $\det P(z) = z^{-k}$ the QMF condition between columns of $P(z)$ holds.*

Proof. Since $P(z)$ is square and paraunitary, then $P^{-1}(z) = P^\sharp(z)$.

Start with the inverse of $P(z)$:

$$P^{-1}(z) = \frac{1}{\det P(z)} \begin{bmatrix} g_o(z) & -g_e(z) \\ -h_o(z) & h_e(z) \end{bmatrix} = z^k \begin{bmatrix} g_o(z) & -g_e(z) \\ -h_o(z) & h_e(z) \end{bmatrix}.$$

On the other hand,

$$P^\sharp(z) = \begin{bmatrix} \tilde{h}_e(z) & \tilde{h}_o(z) \\ \tilde{g}_e(z) & \tilde{g}_o(z) \end{bmatrix}.$$

Equating $P^{-1}(z) = P^\sharp(z)$, we get:

$$z^k g_o(z) = \tilde{h}_e(z), \quad -z^k g_e(z) = \tilde{h}_o(z).$$

Therefore

$$g_o(z) = z^{-k} \tilde{h}_e(z), \quad g_e(z) = -z^{-k} \tilde{h}_o(z).$$

Using the identity (1) for both g and h we can rewrite the above as

$$g(z) = z^{-(2k+1)} \tilde{h}(-z^{-1}) \quad (3)$$

This is exactly the QMF condition for the pair of filters h and g . □

Thus it is enough for us to compute h , because g can be derived using (3). To extract vector $\begin{bmatrix} h_e(z) \\ h_o(z) \end{bmatrix}$ we can multiply $P(z) \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, which gives us the following decomposition:

$$\begin{bmatrix} h_e(z) \\ h_o(z) \end{bmatrix} = \begin{bmatrix} 1 & \alpha_0 \\ -\alpha_0 & 1 \end{bmatrix} \prod_{k=1}^{K-1} \begin{bmatrix} 1 & 0 \\ 0 & z^{-1} \end{bmatrix} \begin{bmatrix} 1 & \alpha_k \\ -\alpha_k & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Note that:

$$\begin{vmatrix} 1 & \alpha_k \\ -\alpha_k & 1 \end{vmatrix} = 1 + \alpha_k^2 \neq 1$$

So we normalize the matrix by the factor of $\frac{1}{\sqrt{1+\alpha_k^2}}$ and introduce new parameters θ_k such that $\alpha_k = \tan \theta_k$. This gives us the following decomposition:

$$\begin{bmatrix} h_e(z) \\ h_o(z) \end{bmatrix} = \begin{bmatrix} \cos \theta_0 & \sin \theta_0 \\ -\sin \theta_0 & \cos \theta_0 \end{bmatrix} \prod_{k=1}^{K-1} \begin{bmatrix} 1 & 0 \\ 0 & z^{-1} \end{bmatrix} \begin{bmatrix} \cos \theta_k & \sin \theta_k \\ -\sin \theta_k & \cos \theta_k \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Which can be also rewritten as:

$$\begin{bmatrix} h_e(z) \\ h_o(z) \end{bmatrix} = \prod_{k=0}^{K-1} R(\theta_k) \begin{bmatrix} 1 & 0 \\ 0 & z^{-1} \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Moreover, since each matrix in the above decomposition is paraunitary, product of them is still paraunitary, thus for any choice of $\{\theta_k\}_{k=0}^{K-1}$ the matrix $P(z)$ is paraunitary.

When the wavelet that is factorized as above, we can find a derivative with respect to all θ_k , this gives us a differentiable parametrization of filters with constraint on filters to have a perfect reconstruction and being orthogonal (i.e. their polyphase matrix is paraunitary).

To summarize, any polyphase matrix of a wavelet can be decomposed into a sequence of rotations and delays. Product of any such sequence produces paraunitary matrix and it has a derivative with respect to angles of rotations. This is a perfect setup for gradient descent over orthogonal filters with perfect reconstruction.

5 Tree structure

We have described one-level wavelet transform in section 3, but we extend it to multi-level decomposition by applying the same operations to one or more subbands recursively. This gives us a 4-ary tree of subbands, where each node corresponds to a subband and has four children corresponding to the result of applying one-level wavelet transform to that subband. The root of the tree is the original image, and the leaves are the smallest subbands obtained after several levels of decomposition.

Since we implement lossy compression, we will apply a projection M to the coefficients after the multi-resolution transformation. This projection zeroes-out coefficients in some leaves of the tree (we call them *Drop*) or leaves them unchanged (we call them *Keep*).

We also allow to use different wavelet filters for each decomposition level. Thus each inner node (including root of the tree) has a wavelet associated with it.

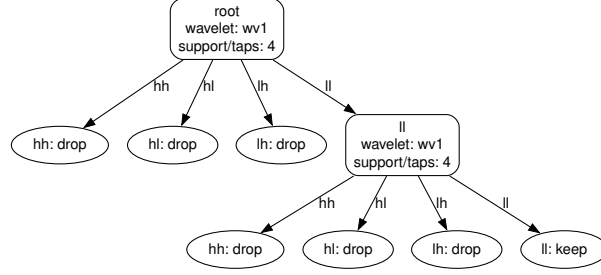


Figure 1: Example of a tree structure

Definition 5.1. Let $\mathcal{T}$ be a set of all possible trees of subbands with associated wavelets (without parameters θ) and projection M .

Definition 5.2. Let $\Phi(T, \mathcal{R}) : \mathcal{T} \rightarrow \mathcal{T}$ be a mutation function that for each $T \in \mathcal{T}$ does one step from the following list:

- Replacing a leaf from *Keep* to *Drop* or vice versa;
- Swapping two leaves;
- Replacing a leaf with a node (wavelet - random existing uniformly or new with fixed probability)
- Replacing a node with a leaf (*Keep/Drop* is chosen with probability proportional to the number of *Keep/Drop* leaves)
- Replacing a wavelet in a node with another wavelet (random existing uniformly or new with fixed probability)

In this case $\mathcal{R}$ is a hidden random variable that defines the choice of mutation.

6 Compression and optimization of angles

Set of angles $\theta = \{\theta_k\}_{k=0}^{K-1}$ as a parameters of the model, this set uniquely (up to a period of 2π of each angle) generates wavelet, which has a derivative with regard to each θ_k . The detailed parametrization is described in Parametrization of orthogonal wavelets.

Definition 6.1. Let $F(X, T, \Theta)$ be a function that applies multi-level wavelet transform defined by tree T with wavelets generated by Θ to the image X , then applies projection M to the coefficients and then applies inverse wavelet transform to get the reconstructed image:

$$F(X, T, \Theta) = (P^{-1} \circ M \circ P)\{X\}$$

Since M is the projection, $F(X, T, \Theta) \neq X$ for some of the images X , but we want it to be as close as possible, thus we use a reconstruction loss:

$$J(T, \Theta) = \sum_{i=1}^m \|F(X_i, T, \Theta) - X_i\|_F^2$$

using the Frobenius norm to measure the distance between the original and reconstructed images. This is equivalent to MSE loss with element-wise comparison.

We are interested in finding $\arg_{\Theta} \min J(T, \Theta)$. We will use gradient descent to minimize it, but specifically an Adam optimizer[8] to speed up convergence and avoid local minima.

7 Tree optimization

To optimize the tree structure, we will use a simulated annealing algorithm. Our goal is to maximize PSNR (Peak Signal-to-Noise Ratio) of the reconstructed images.

Definition 7.1 (PSNR). Let X be an original image and $\hat{X}$ be a reconstructed image. Then PSNR is defined as follows:

$$\text{PSNR}(X, \hat{X}) = 10 \cdot \log_{10} \left(\frac{\text{MAX}^2}{\text{MSE}(X, \hat{X})} \right)$$

in our case $\text{MAX} = 2$, because images are in $[-1, 1]$ range, and $\text{MSE}(X, \hat{X}) = \frac{1}{N^2} \|X - \hat{X}\|_F^2$

Note that maximizing PSNR is equivalent to minimizing MSE, thus $\arg_{\Theta} \max \text{PSNR} = \arg_{\Theta} \min J(T, \Theta)$.

Definition 7.2. Let $\Omega(T) = -\max_{\Theta} \text{PSNR}(T, \Theta)$ be the negative of the optimal value of the PSNR for a given tree T after optimizing angles Θ . $\Omega(T)$ is also called an energy of the T .

Now our task is to find $\arg_T \min \Omega(T)$. We will use simulated annealing to solve this problem. The algorithm is described in algorithm 1.

Input: Initial tree $T_0 \in \mathcal{T}$, energy $\Omega : \mathcal{T} \rightarrow \mathbb{R}$, initial temperature τ_0 , final temperature $\tau_{\min}$, cooling rate $\alpha \in (0, 1)$, batch size B , batches per temperature M

Output: Approximate minimizer T^* and energy Ω^*

```

 $T \leftarrow T_0;$ 
 $E \leftarrow \Omega(T);$ 
 $T^* \leftarrow T;$ 
 $\Omega^* \leftarrow E;$ 
 $\tau \leftarrow \tau_0;$ 
while  $\tau > \tau_{\min}$  do
  for  $m = 1, \dots, M$  do
    Sample neighbours
     $\tilde{T}_1, \dots, \tilde{T}_B \leftarrow \Phi(T, \mathcal{R}_1), \dots, \Phi(T, \mathcal{R}_B)$ 
    Evaluate
     $\tilde{E}_b \leftarrow \Omega(\tilde{T}_b), \quad b = 1, \dots, B$ 
    for  $b = 1, \dots, B$  do
       $p_b = \begin{cases} 1, & \text{if } \tilde{E}_b < E \\ \exp\left(-\frac{\tilde{E}_b - E}{\tau}\right), & \text{otherwise} \end{cases}$ 
      Draw  $X_b \sim \text{Bernoulli}(p_b);$ 
      if  $X_b = 1$  then
         $T \leftarrow \tilde{T}_b;$ 
         $E \leftarrow \tilde{E}_b;$ 
      end
      if  $E < \Omega^*$  then
         $T^* \leftarrow T;$ 
         $\Omega^* \leftarrow E;$ 
      end
    end
  end
   $\tau \leftarrow \alpha\tau;$ 
end
return  $(T^*, \Omega^*);$ 

```

Algorithm 1: Simulated Annealing over Trees

8 Dataset

We will train our model on the Celeba[9] dataset consisting of 200 000 face images. This makes model specialize on face images and generalize poorer, but training will be computationally available for us.

9 Training details

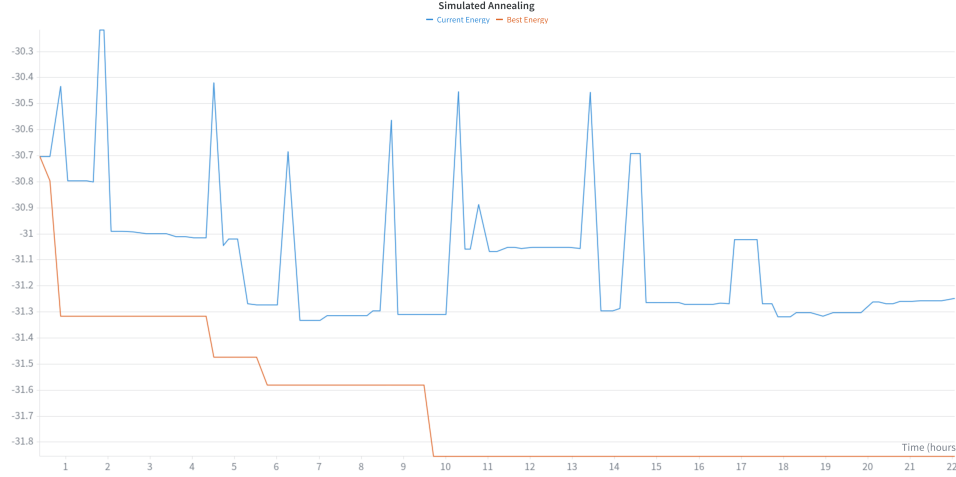


Figure 2: Training curves for the simulated annealing

We train our model on two *Nvidia RTX 3090* GPUs for 22 hours. The training curve is shown on fig. 2. The best tree has achieved PSNR of 31.85 dB on the validation set. The best tree is shown in fig. 3.

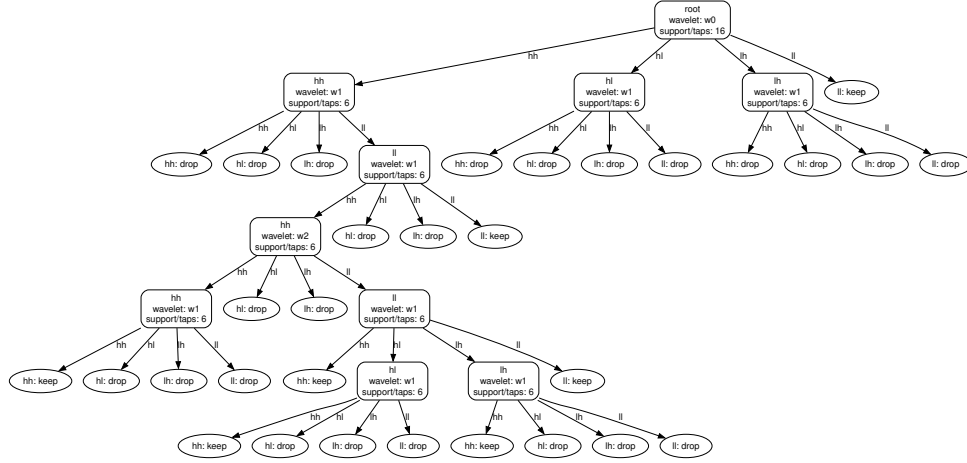


Figure 3: Best tree found by the simulated annealing

The mother wavelet functions for the best tree are shown on fig. 4.

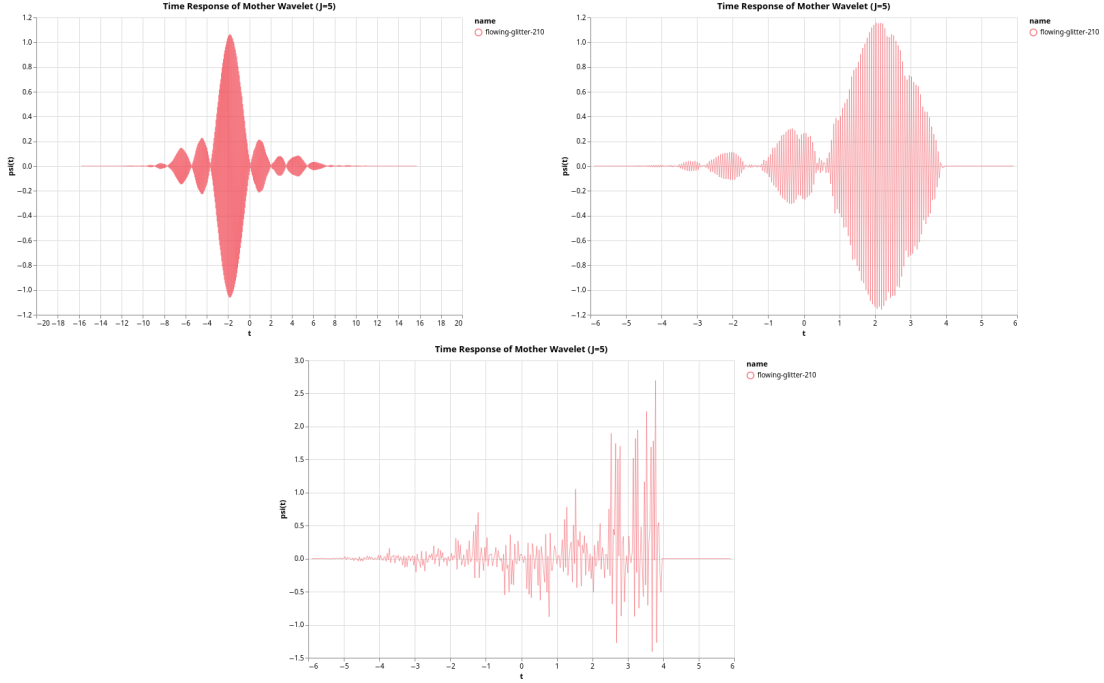


Figure 4: Mother wavelet functions for the best tree

10 Benchmarking and results

To test the model and compare it against existing compression algorithms we will compare PSNR-vs-bpp curve. The result is shown on fig. 5.

Definition 10.1 (bpp). Let bytes be the number of bytes required to store the compressed representation of the image and points be the number of pixels in the original image. Then bits per pixel (bpp) is defined as follows:

$$\text{bpp} = \frac{\text{bytes} \cdot 8}{\text{points}}$$

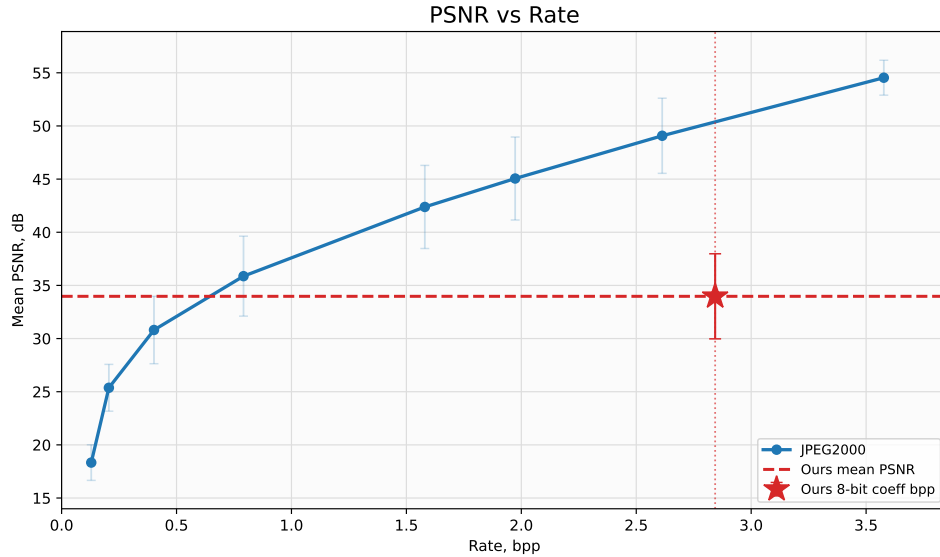


Figure 5: PSNR-vs-bpp curve for our model and JPEG2000[3]

We also show some reconstructed images for different bpp values on fig. 6.

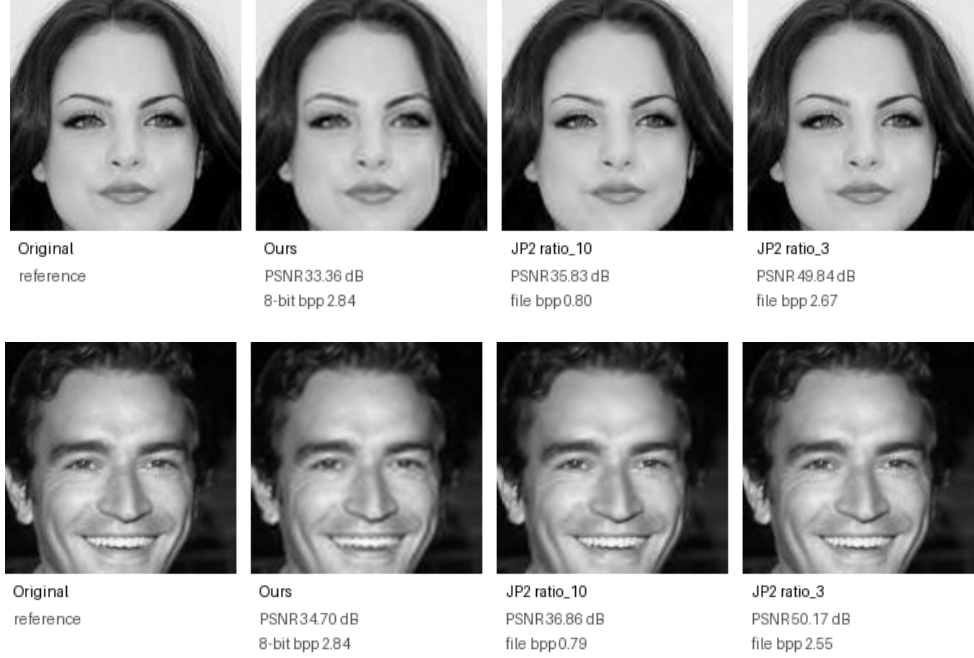


Figure 6: Sample images from the CelebA dataset

Our method reconstructs images with 33.97 dB mean PSNR while keeping 35.54% of coefficients quantized to 8 bits. JPEG2000 achieves similar or better PSNR at lower real file bpp, because it also uses quantization, code-block coding, and entropy coding.

References

- [1] M. W. Frazier, *An Introduction to Wavelets Through Linear Algebra*. Springer, Undergraduate Texts in Mathematics.
- [2] International Telecommunication Union (ITU), *Information Technology – Digital Compression and Coding of Continuous-Tone Still Images (JPEG)*, ITU-T Recommendation T.81, Sept. 1992. Available: <https://www.w3.org/Graphics/JPEG/itu-t81.pdf>
- [3] International Telecommunication Union (ITU), *Information Technology – JPEG 2000 Image Coding System: Core Coding System*, ITU-T Recommendation T.800, Aug. 2002. Available: https://www.itu.int/rec/dologin_pub.asp?lang=e&id=T-REC-T.800-200208-S!!PDF-E&type=items
- [4] U. B. Sahin and F. Kamisli, “Image Compression with Learned Lifting-Based DWT and Learned Tree-Based Entropy Models,” *arXiv preprint arXiv:2212.03616*, 2022. Available: <https://arxiv.org/abs/2212.03616>
- [5] P. P. Vaidyanathan, “Multirate Systems and Filter Banks“. Pearson, 1993.
- [6] M. Vetterli and Didier Le Gall, “Perfect reconstruction FIR filter banks: some properties and factorizations,” *IEEE Transactions on Acoustics, Speech, and Signal Pro-*

cessing, vol. 37, no. 7, pp. 1057–1071, Jul. 1989, doi: <https://doi.org/10.1109/29.32283>.

- [7] J. Ansel et al., “PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '24)*, 2024, doi: <https://doi.org/10.1145/3620665.3640366>.
- [8] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” *International Conference on Learning Representations (ICLR)*, 2015. Available: <https://arxiv.org/abs/1412.6980>.
- [9] Z. Liu et al., “CelebFaces Attributes (CelebA) Dataset,” Available: <http://mmlab.ie.cuhk.edu.hk/projects/CelebA.html>.
- [10] A. Kryvyi, N. Lenyk, O. Kostiuk, L. Konovalov, GitHub repository of the project. Available: <https://github.com/draklowell/learnable-wavelets/>.